

Developer Manual

Team:

Tejaswi Gadde: 2884829

Harika Ummaneni: 2895833

1. INTRODUCTION

1.1 Purpose of the Developer Manual

This Developer Manual provides complete technical documentation for maintaining, extending, and deploying the Job Board Application. It explains the system architecture, folder structure, codebase organization, database schema, development setup, configuration details, and the internal workflow of the backend and frontend systems.

1.2 Audience

This document is intended for:

- Developers working on the Job Board project
- Students learning Flask/SQLAlchemy development
- Testers and maintainers
- Project evaluators

1.3 Technology Stack

- **Frontend:** HTML, CSS, JavaScript (Jinja2 Templates)
- **Backend:** Python (Flask Framework)
- **Database:** SQLite
- **ORM:** SQLAlchemy
- **Version Control:** GitHub
- **Deployment:** Localhost / Render / Azure

2. SYSTEM OVERVIEW

The Job Board Application connects **Job Seekers**, **Employers**, and **Admin** users. It includes:

Job Seeker Features

- Register & Login
- Upload Resume
- Search for Jobs
- Apply for Jobs
- Track Application Status

Employer Features

- Register & Login
- Create/Edit/Delete Jobs
- View Applicants
- Update Applicant Status

Admin Features

- View all users
- View all jobs
- Monitor applications
- Basic platform statistics

3. PROJECT STRUCTURE

job-board-app/

|

|— backend/

| |— app.py → Main Flask application

| |— models.py → ORM models (Admin, Employer, JobSeekers, Jobs, Applications)

|

|— templates/ → Frontend HTML templates

| |— login.html

```
| ├── register.html
| ├── index.html
| ├── employer_jobs.html
| ├── employer_post_job.html
| ├── edit_jobs.html
| ├── admin_dashboard.html
| ├── job_listings.html
| └── my_applications.html
|
| ├── static/
| │   └── css/
| │       └── style.css
|
| ├── uploads/           → Uploaded resumes
|
| ├── job_board.db       → SQLite Database
|
| └── run.py / wsgi.py   → App launcher
```

4. DEVELOPMENT ENVIRONMENT SETUP

4.1 Requirements

- Python 3.10 or higher
- SQLite installed (comes default with Python)
- pip (Python package manager)

4.2 Install Dependencies

```
pip install -r requirements.txt
```

4.3 Running the project locally

```
export FLASK_APP=run.py
```

```
flask run
```

or on Windows:

```
set FLASK_APP=run.py
```

```
flask run
```

App will start at:

```
http://127.0.0.1:5000
```

5. DATABASE DESIGN

The application uses **SQLite + SQLAlchemy ORM**.

5.1 Tables

1. Admin

Field	Type
-------	------

id	Integer, PK
----	-------------

username	String
----------	--------

password	String (hashed)
----------	-----------------

2. Employer

Field	Type
-------	------

id	Integer
----	---------

name	String
------	--------

Field	Type
email	String
password	String
company	String

3. JobSeeker

Field	Type
id	Integer
name	String
email	String
password	String
resume	String

4. Job

Field	Type
id	Integer
title	String
description	String
location	String
salary	Float
category	String
employer_id	Integer (FK → Employer.id)

5. Applications

Field	Type
id	Integer
job_id	Integer (FK → Job.id)
seeker_id	Integer (FK → JobSeeker.id)
status	String

6. BACKEND LOGIC (FLASK + SQLALCHEMY)

6.1 App Initialization

- Loads templates and static assets
- Initializes database
- Connects to SQLite using a fixed path
- Creates admin user automatically

6.2 Routes Overview

Authentication

- /register
- /login
- /logout

Job Seeker

- /apply/<job_id>
- /my-applications
- /job-listings

Employer

- /employer/jobs

- /employer/post-job
- /employer/edit-job/<id>
- /employer/delete-job/<id>
- /employer/view-applications/<job_id>

Admin

- /admin/dashboard

7. FRONTEND (TEMPLATES)

Templates use **Jinja2** syntax:

```
{% for job in jobs %}
```

```
<h3>{{ job.title }}</h3>
```

```
{% endfor %}
```

Key UI Pages

- Clean registration form
- Login form with role selector
- Employer dashboard
- Job listing table
- Admin statistics dashboard

8. VALIDATION RULES

8.1 Email Validation (Frontend + Backend)

Frontend HTML5 Pattern

```
<input type="email" name="email"
```

```
pattern="^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$"
```

```
required>
```

Backend Python Validation

```
import re

pattern = r"^[\\w\\.]+@[\\w\\.]+\\.\\w+$"

if not re.match(pattern, email):

    flash("Invalid email format", "danger")
```

8.2 Password Rules

- Min 8 chars
- At least **1 letter**
- At least **1 number**
- At least **1 special char**

9. DEPLOYMENT GUIDE

9.1 Local Deployment

Simply run:

```
flask run
```

9.2 Cloud Deployment (Render/Azure)

You need:

- requirements.txt
- start command
- Correct database path

For free hosting, **local SQLite path will NOT work online**, because cloud resets storage.

10. COMMON ERRORS & FIXES

Error	Cause	Fix
No data in DB	Wrong DB path	Verify DB path printed in console
Resume upload not stored uploads/ missing		Create folder manually

Error	Cause	Fix
Admin login fails	No default admin created	Delete DB & restart app
Jobs not visible	employer_id mismatch	Check session user_id

11. EXTENDING THE SYSTEM

Possible Enhancements:

- Password reset
- Email notifications
- Profile dashboard
- Cloud-based PostgreSQL
- JWT Authentication
- React frontend

12. MAINTENANCE GUIDELINES

- Always test new routes
- Keep a backup of job_board.db
- Do not modify database manually
- Validate all user input
- Use debug=True only locally

13. VERSION CONTROL RULES

- All code must be pushed to GitHub
- Use meaningful commit messages
- Keep individual feature branches
- Merge only after testing

14. CONCLUSION

The Job Board Application is built with clean architecture, modular design, and scalable components. This Developer Manual provides all the necessary information to:

- Understand the system
- Extend or fix the code
- Deploy locally or online
- Maintain database and logic